CSE 209 — Computer Architecture

Graphics and Computing GPUs · Final Integration of the RS Part

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Rifat Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, bit layouts and hand-drawn datapath sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — GPU questions by year + what is out of scope
Lesson 33 — GPUs and Heterogeneous CPU–GPU Systems — evolution, CPU vs GPU, historical PC, discrete GPU, UMA, memory access
Lesson 34 — Graphics Pipeline and the Unified GPU Architecture — shaders, unified processor array, SMs, SPs, SIMT warps
Lesson 35 — CUDA: Kernels, Threads, Blocks and Grids — SPMD, SAXPY, memory levels, barriers, mapping to SMs and SPs
Lesson 36 — Final Integration: Whole RS Part — eight great ideas, formula sheet, diagram checklist, mixed TF practice
Quick Notes — GPUs — merged revision notes + key diagrams

> **NOTE** Slide numbers refer to **Graphics and Computing GPUs — Slides** (35 slides). A few details the TF answers need (Tesla SM numbers, warps/SIMT, how blocks are assigned to SMs) come from the accompanying **chapter** and are marked "(chapter)". Lesson 36 ties together all three RS note files: Chapter 4 Part 1, Chapter 4 Part 2 and Chapter 5.

Exam Map — What the TF Papers Ask

The GPU part is small, and its TF questions **repeat word for word**.

Session (paper)	GPU questions
2023-24 (CSE 209)	Q4(a) **Block diagrams of contemporary PCs with Intel and AMD CPUs** connecting a separate GPU; **how do the CPU and GPU access each other's memory?** · Q4(c) **Kernel, grid, thread block** in CUDA; **how they map to SP cores and SMs** of NVIDIA's unified GPU architecture
2021-22 (CSE 305)	Q4(b) and Q4(c) — **exactly the same two questions** (10 marks each)
2020-21 (CSE 305)	Q1 — match the statement "As CPUs have hit their physical performance barrier due to power limit issues, GPUs have started gaining increased popularity" to one of the **eight great ideas**

EXAM FOCUS **Learn two answers perfectly:** (1) the Intel/AMD discrete-GPU block diagrams + memory access, and (2) kernel/grid/thread block + mapping to SMs and SPs. Together they are worth about 20 marks and appeared, unchanged, in both 2023-24 and 2021-22.

NOTE Papers also contain SISD/SIMD/MIMD, shared-memory multiprocessors, hardware multithreading (coarse/fine/SMT) and vector vs array processors (2022-23 CSE 305 Q4(c), 2021-22). These belong to the **multiprocessor chapter**, which is not among the RS materials provided, so they are not covered here.

Lesson 33 — GPUs and Heterogeneous CPU–GPU Systems

Slides: 2–4, 10–19 and 24–25

EXAM FOCUS **2023-24 Q4(a) = 2021-22 Q4(b):** "Show with block diagrams how contemporary PCs with Intel and AMD CPUs connect a separate GPU. How do the CPU and GPU access each other's memory?" **2020-21 Q1:** why GPUs became popular.

1. What Is a GPU? (slides 2–3)

- A **Graphics Processing Unit** is a processor optimized for 2D/3D graphics, video, visual computing and display.
- Modern GPUs are **highly parallel, highly multithreaded multiprocessors**. They provide **two to three orders of magnitude** more thread and data parallelism than CPUs.
- **Visual computing:** a mix of graphics processing and computing that allows real-time interaction with computed objects.

Evolution:

Period	Graphics hardware
Pre-1990s	Simple **VGA controllers** — essentially memory controllers connected to DRAM
1990s	3D acceleration added: triangle setup, rasterization, texture mapping
2000	The term **"GPU"**: a single chip containing almost the whole workstation graphics pipeline
Modern	**Massively parallel programmable processors** — hundreds of cores, thousands of threads, programmable in C/C++

2. CPU vs GPU: a Heterogeneous System (slides 4, 24)

A **heterogeneous system** combines different processor types, such as a CPU and a GPU, and splits work according to their strengths.

	CPU	GPU
Kind	Multicore, mainly **serial** processor	**Massively parallel manycore** processor
Optimized for	**Latency** of one thread	**Throughput** of many threads
Cores	Few, complex	Many, simple, highly multithreaded
Transistor budget	Large caches and control overhead	Mostly **computation**; less cache and overhead
Best at	Sequential, control-heavy code	Fine-grained data parallelism, floating-point-heavy work

Why GPUs became popular (2020-21 Q1): CPU clock rates hit the **power wall**, so more performance has to come from **parallelism**. GPUs supply massive parallelism and their parallelism doubles roughly every **12–18 months** (Moore's Law) → great idea: **performance via parallelism** (helped by Moore's Law).

3. System Architectures (slides 11–17)

Historical PC (circa 1990)

A **VGA controller** drives the display from **framebuffer memory**. It sits on the low-bandwidth **PCI bus**. The **North Bridge** handles the high-bandwidth CPU–memory interfaces; the **South Bridge** handles legacy I/O (LAN, UART, audio). No programmable GPU existed.

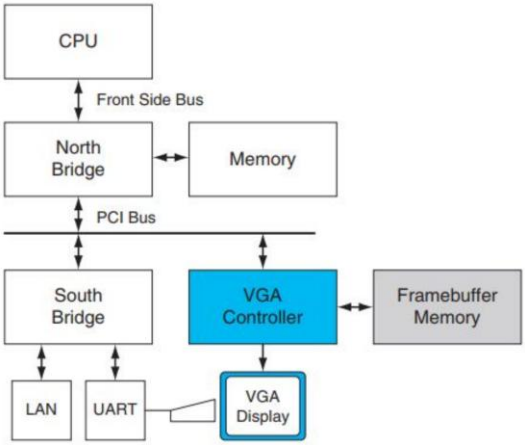


Figure 33.1 — Historical PC: the VGA controller drives the display from framebuffer memory (slide 13)

Contemporary PCs with a discrete GPU (slides 14–15) ★

A **discrete GPU** and the CPU each have their **own memory subsystem**.

	(a) Intel CPU	(b) AMD CPU
CPU link	CPU → **Front Side Bus** → North Bridge	**North Bridge (memory controller) on the CPU die**, reached by an internal bus
System memory	DDR2 memory on the North Bridge (128-bit, 667 MT/s)	DDR2 memory on the on-die North Bridge (128-bit, 667 MT/s)
GPU link	GPU on a **×16 PCI-Express link** to the North Bridge	GPU on a **×16 PCI-Express link** to the **chipset**, which connects to the CPU by **HyperTransport**
Other	South Bridge on a ×4 PCIe-derivative link	—
GPU memory	Dedicated **GPU memory**; GPU drives the display	Same

A ×16 PCIe link gives a peak of **16 GB/s (8 GB/s in each direction)**.

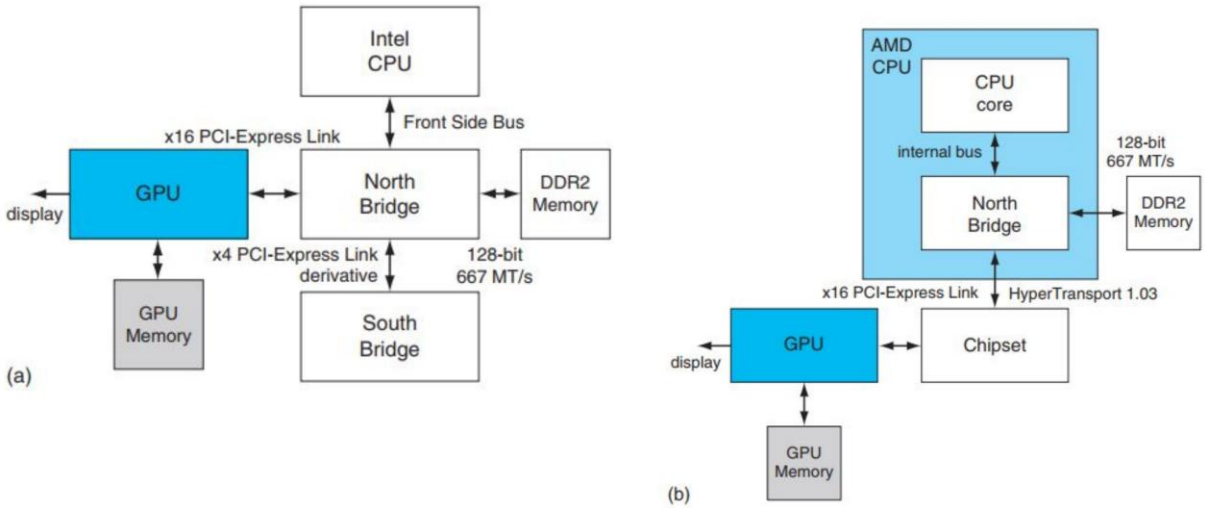


Figure 33.2 — Contemporary PCs with (a) Intel and (b) AMD CPUs (slide 15)

How do the CPU and GPU access each other's memory? (slides 14, 17)

- Both **can** access each other's memory, but with **much less bandwidth** than access to their own directly attached ("local") memory.
- There are **multiple address spaces**. The GPU reaches **its own local memory and the CPU's system memory through virtual addresses** translated by an **MMU on the GPU**; the **OS kernel manages the GPU's page tables**.

- A system physical page is accessed with **coherent or noncoherent PCI-Express transactions**, chosen by an **attribute in the GPU's page table**.
- The CPU accesses the GPU's local memory through an **address range ("aperture") in the PCI-Express address space**.

Other variants (slides 16, 18–19)

- **UMA (unified memory architecture):** CPU and GPU **share system memory**; no GPU memory → cheaper, but **lower performance** (limited by system-memory bandwidth, higher latency).
- GPU **integrated** with the North Bridge (Intel) or chipset (AMD), with or without dedicated graphics memory.
- **Multi-GPU:** 2–4 GPUs in parallel, e.g. NVIDIA **SLI**.
- **Consoles** (PS3, Xbox 360): like PCs but with **customized internal buses**; constant capability for 5+ years, re-implemented on newer silicon to cut cost.
- **Interfaces:** applications call **OpenGL / Direct3D**; the **graphics driver** sends commands, programs and data to the GPU over PCIe (earlier **AGP**).

Practice — Lesson 33

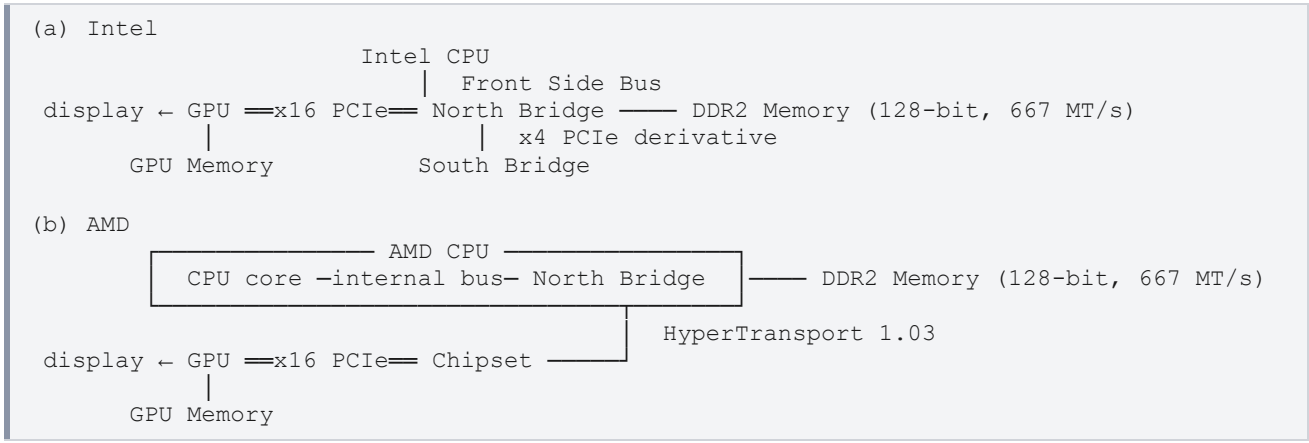
Actual — 2023-24 Q4(a) and 2021-22 Q4(b). Part (b): Actual — 2020-21 Q1.

(a) Show with block diagrams how contemporary PCs with Intel and AMD CPUs connect a separate GPU. How do the CPU and GPU access each other's memory?

(b) "As CPUs have hit their physical performance barrier due to power limit issues, GPUs have started gaining increased popularity among the programming community." Which great idea is behind this? Justify.

✓ Solution

(a) Draw both diagrams of Figure 33.2:



EXAM-READY ANSWER In both systems the GPU is a discrete device with its own GPU memory, attached by a 16-lane PCI-Express link (16 GB/s peak, 8 GB/s each way): to the North Bridge in the Intel system (the CPU reaches the North Bridge over the front side bus) and to the chipset in the AMD system (whose memory controller/North Bridge is integrated on the CPU die and linked to the chipset by HyperTransport). Each processor can access the other's memory, but with less bandwidth than its local memory. The GPU uses virtual addresses translated by its own MMU, with page tables managed by the OS kernel, to reach both its local memory and system memory; a page-table attribute chooses coherent or noncoherent PCIe transactions. The CPU accesses GPU memory through an aperture — an address range in the PCI-Express address space.

(b) Performance via parallelism. Power limits stopped further increases in single-CPU clock rate, so performance must come from doing many operations at once. GPUs are massively parallel, highly

multithreaded processors (two to three orders of magnitude more parallelism than CPUs, doubling every 12–18 months), so programmers use them to get performance through parallelism.

Lesson 34 — Graphics Pipeline and the Unified GPU Architecture

Slides: 5–6, 20–24 · SM/warp details from the chapter

> **EXAM FOCUS** Needed for the **"map to SP cores and SMs"** half of 2023-24 Q4(c) / 2021-22 Q4(c): you must know what an SM and an SP are.

1. The Logical Graphics Pipeline (slides 5, 20, 26)

```
Input Assembler → Vertex Shader → Geometry Shader → Setup & Rasterizer → Pixel Shader → Raster Operations / Output Merger
```

Stage	Type	Job
Input Assembler	Fixed	Collects vertices and groups them into primitives (triangles, polygons)
Vertex shader	Programmable	Per-vertex work: 3D → screen position, lighting
Geometry shader	Programmable	Per-primitive work; can **add or drop** primitives
Setup & Rasterizer	Fixed	Generates pixel fragments
Pixel shader	Programmable	Per-fragment work: texturing, colouring
Output Merger (ROP)	Fixed	Z-buffer depth test, colour blending

Shading languages (slide 6): HLSL and Cg — C-like but without pointers, file I/O or recursion; inputs are interpolated implicitly; many instances run **in parallel as independent threads** because they work on independent data with no side effects.

2. Unified Architecture (slides 21–23)

Older GPUs had **separate hardware** for each shader type. A **unified** GPU runs vertex, geometry, pixel shaders **and general computing** on the **same array of programmable processors**; the logical pipeline **recirculates** through these physical processors. The array is tightly integrated with fixed-function units (texture filtering, video decoding, anti-aliasing).

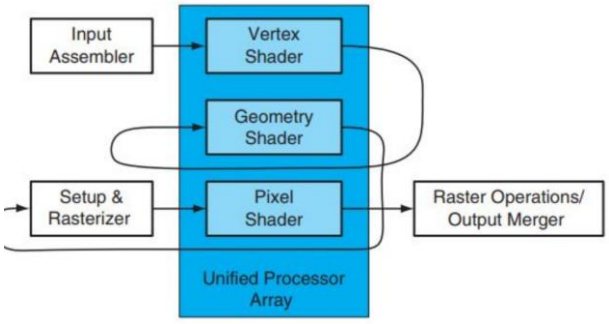


Figure 34.1 — Logical pipeline mapped onto a unified processor array (slide 21)

NVIDIA GeForce 8800 (Tesla) — numbers to remember (slide 22)

- **112 streaming processor (SP) cores** organized into **14 streaming multiprocessors (SMs)**.
- Each **SM** = **8 SP cores**, **2 special function units (SFUs)**, a **shared memory** (16 KB), instruction and constant caches, and a **multithreaded instruction issue unit**.
- Each SP is highly multithreaded — **96 concurrent threads** with their state in hardware (so an SM holds 8 × 96 = **768 threads**).
- The SMs connect through an interconnection network to the DRAM partitions; pairs of SMs share a texture unit (texture/processor cluster, TPC).
- The design **scales** to different markets by changing the number of SMs and memory partitions.

NOTE The slides use the **112 SP / 14 SM** configuration. The chapter's later GeForce 8800 figure shows the full chip with **128 SPs in 16 SMs** (8 TPCs). Quote the slide numbers unless the question gives others.

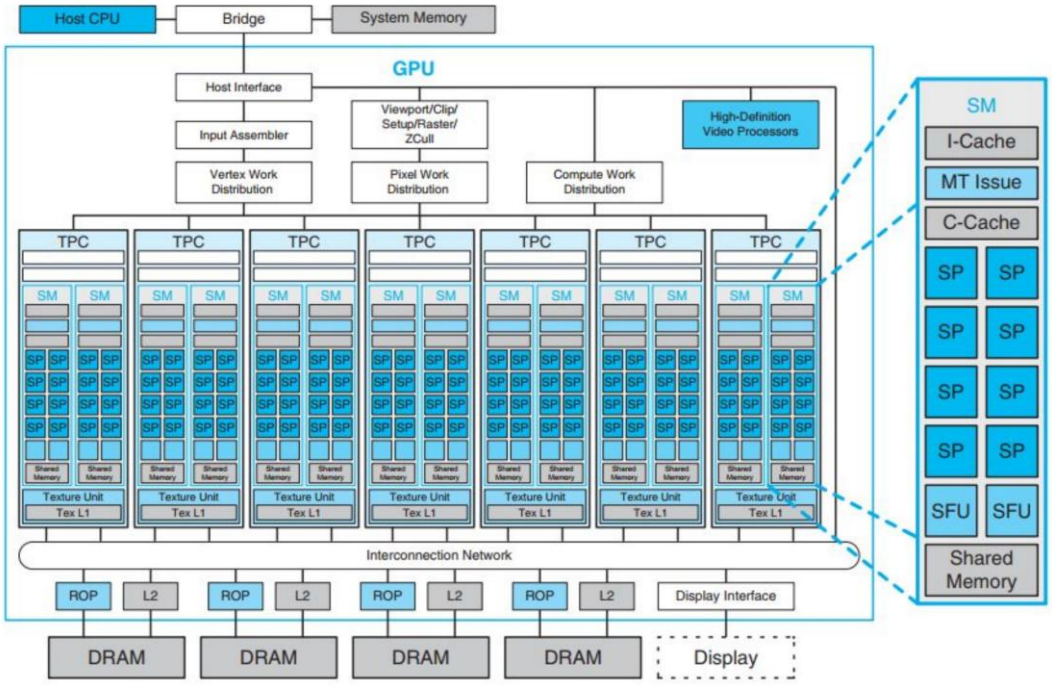


Figure 34.2 — Basic unified GPU architecture: 14 SMs × 8 SPs (slide 23)

3. How an SM Runs Threads: SIMT and Warps (chapter)

- The SM uses a **SIMT (single-instruction, multiple-thread)** architecture: threads are managed in groups of **32 called warps**.
- Each issue cycle the multithreaded issue unit **picks a ready warp** and issues one instruction to all its active threads; a warp instruction runs as 4 sets of 8 threads on the **8 SPs over 4 cycles**.
- Switching between warps hides memory latency with **zero scheduling overhead**.
- Full efficiency when all threads of a warp follow the **same path**; if they **diverge** (if/else), the paths execute one after the other.

MEMORIZE **Design philosophy (slides 24, 35):** fine-grained data parallelism (one thread per pixel/vertex/result), **high throughput** (millions of threads per frame), **transparent scalability**, and **floating-point intensity**.

Lesson 35 — CUDA: Kernels, Threads, Blocks and Grids

Slides: 7–9, 27–34 · mapping details from the chapter

EXAM FOCUS **2023-24 Q4(c) = 2021-22 Q4(c):** "What do you mean by kernel, grid, and thread block of the CUDA paradigm? Briefly describe how they map to the streaming processor (SP) cores and streaming multiprocessors (SM) of the unified GPU architecture of NVIDIA."

1. From GPGPU to CUDA (slide 7)

- **GPGPU:** general-purpose computing through **graphics APIs** and the graphics pipeline — awkward and limiting.
- **GPU computing:** use the GPU through a **parallel programming language and API**, without the graphics model.
- **CUDA (Compute Unified Device Architecture):** NVIDIA's scalable parallel programming model — a minimal extension of **C/C++**.

2. The CUDA Model (slides 8, 27–28)

SPMD style (single program, multiple data): write code for **one thread**; many threads run it in parallel.

Term	Meaning
Kernel	A C function / program **for one thread**, executed by many threads in parallel (declared `__global__`)
Thread	Finest unit of work; unique ID **threadIdx** inside its block
Thread block	Concurrent threads that **cooperate** through **barrier synchronization** (`__syncthreads()`) and **shared memory**; ID **blockIdx**, size **blockDim**
Grid	A set of **independent** thread blocks that all execute the **same kernel**

Three key abstractions: a hierarchy of thread groups, shared memories, and barrier synchronization.

Decomposition (slides 9, 29): partition the result array into **blocks**, each block into **elements**; parallel multiprocessors compute result blocks, parallel threads compute elements. The number of blocks is set by the **data size**, not the number of processors (**virtualization**).

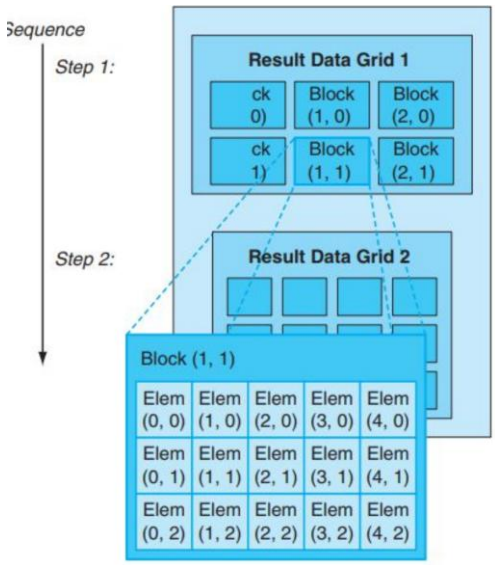


Figure 35.1 — Result data decomposed into a grid of blocks of elements (slide 29)

3. Worked Example: SAXPY (y = a·x + y) (slides 30–31)

```
// Sequential C
void saxpy_serial(int n, float alpha, float *x, float *y)
{
    for (int i = 0; i < n; ++i)
        y[i] = alpha * x[i] + y[i];
}
saxpy_serial(n, 2.0, x, y);

// Parallel CUDA: one thread per loop iteration
__global__
void saxpy_parallel(int n, float alpha, float *x, float *y)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = alpha * x[i] + y[i];
}
int nblocks = (n + 255) / 256;
saxpy_parallel<<<nblocks, 256>>>(n, 2.0, x, y);
```

- Each thread computes **one** result; its global index is **blockIdx.x × blockDim.x + threadIdx.x**.
- `<<<nblocks, 256>>>` launches a grid of nblocks blocks of 256 threads. `(n + 255) / 256` rounds **up**; the `if (i < n)` guard stops the extra threads in the last block.
- Host program (slide 31): allocate device arrays (`cudaMalloc`), copy inputs host → device (`cudaMemcpy … HostToDevice`), **launch the kernel**, copy results device → host, free device memory (`cudaFree`).

Example: n = 1000 → nblocks = (1000 + 255) / 256 = **4** blocks → 1024 threads; threads 1000–1023 do nothing. Thread 5 of block 2 computes element 2 × 256 + 5 = **517**.

4. Memory Levels and Synchronization (slides 32–34)

Level	Shared by	Where
Local memory	One thread (private)	Per-thread
Shared memory	All threads of **one block**	Fast **on-chip** RAM in the SM
Global memory	**All threads** of the application (and the host)	DRAM on the graphics board

- `__syncthreads()` is a **barrier**: no thread of the block continues until all have reached it.
- **Independence rule:** blocks of a grid must be able to run **in any order, in parallel or in series** — no direct communication between blocks (atomic operations on global memory only). This gives **scalability** and avoids deadlock.
- Kernels in sequence are separated by an **inter-kernel synchronization barrier**: kernel G starts after all blocks of kernel F finish (slide 34).

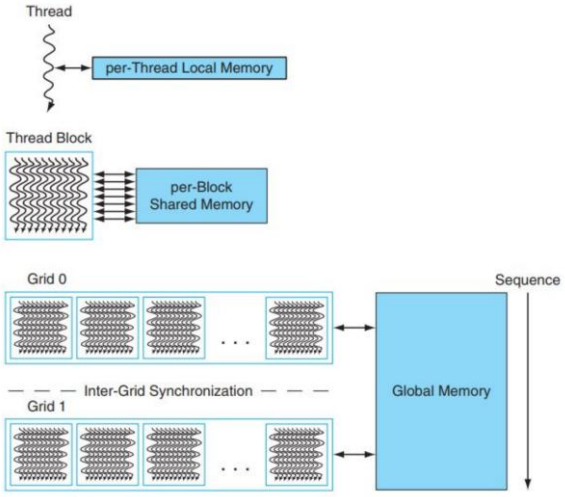


Figure 35.2 — Thread / block / grid and their local / shared / global memories (slide 33)

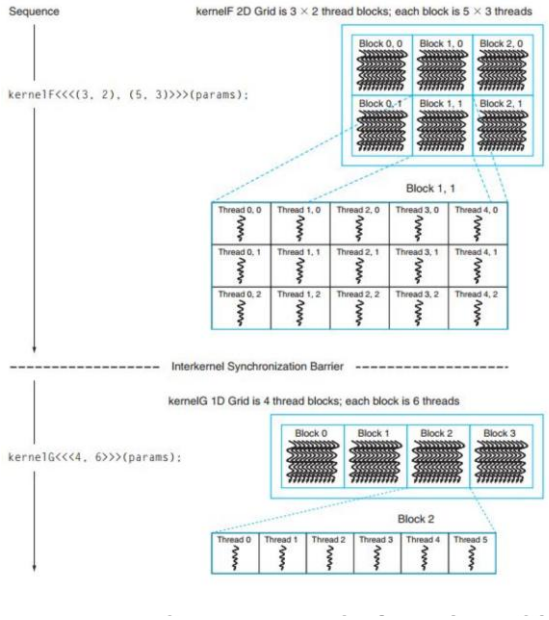


Figure 35.3 — kernelF on a 2D grid (3 × 2 blocks of 5 × 3 threads), barrier, then kernelG on a 1D grid (4 blocks of 6 threads) (slide 34)

5. Mapping to SMs and SP Cores (chapter) ★

CUDA level	Hardware
Grid (one kernel launch)	The **whole GPU** (all SMs). The compute work distribution unit hands its thread blocks to SMs that have free resources; remaining blocks wait and are issued as SMs finish — any order.
Thread block	**One SM.** All threads of a block stay on the same SM, so they can share that SM's **shared memory** and use its **barrier** hardware. An SM can hold **several blocks** at once (up to 768 threads on Tesla). Implemented as a cooperative thread array (CTA) of one or more **warps**.
Thread	**An SP core** (thread processor) with its own registers and execution state. Threads run in **warps of 32**: the SM issues one instruction per warp, executed on its **8 SPs** over 4 cycles; the SM switches warps to hide latency.

Exam Approach

For the 10-mark question, write in this order:

1. Definitions of **kernel, thread block, grid** (one line each, plus threadIdx/blockIdx).

2. A small diagram: grid → blocks → threads (Figure 35.1 / 35.2) with local / shared / global memory.
3. The **mapping table** above, with Tesla numbers (14 SMs × 8 SPs = 112 SPs, 96 threads per SP).
4. One sentence on **scalability:** because blocks are independent and assigned to whatever SMs are free, the same compiled program runs on GPUs with any number of SMs.

TRAP

- A thread block **never** spans two SMs (it would lose shared memory and barriers).
- A grid is not "one SM" — it is spread over all SMs.
- The number of blocks is chosen from the **data size**, not from the number of SMs.

Practice — Lesson 35

Part (a): Actual — 2023-24 Q4(c) and 2021-22 Q4(c). Parts (b)–(c): Constructed.

(a) What do you mean by kernel, grid, and thread block of the CUDA paradigm? Briefly describe how they map to the SP cores and SMs of NVIDIA's unified GPU architecture.

(b) SAXPY is launched for n = 3000 with 256 threads per block. How many blocks and threads are created, how many threads do no work, and which element does thread 17 of block 9 compute?

(c) For kernelF<<<(3, 2), (5, 3)>>> followed by kernelG<<<4, 6>>> (slide 34), how many threads does each kernel create? Can a block of kernelG start before every block of kernelF has finished?

✓ Solution

(a)

EXAM-READY ANSWER A **kernel** is a C/C++ function written for a single thread that is executed in parallel by many threads (SPMD). The threads are organized into **thread blocks**: groups of concurrent threads, each with a threadIdx, that can cooperate through per-block shared memory and barrier synchronization (__syncthreads). A **grid** is the set of independent thread blocks (each with a blockIdx) that execute the same kernel for one launch; its blocks must be able to run in any order.

Mapping on the unified (Tesla) GPU — 14 SMs, each with 8 SP cores, 2 SFUs and a shared memory: a **grid** is distributed over the whole processor array; the work distribution unit assigns its blocks to SMs with free resources and issues the remaining blocks as SMs become free. Each **thread block** executes entirely on **one SM**, using that SM's shared memory and barrier hardware; an SM can run several blocks concurrently (up to 768 threads). Each **thread** runs on an **SP core** with its own registers and state; the SM's multithreaded issue unit schedules threads in warps of 32, issuing one instruction per warp across its 8 SPs. Because blocks are independent, the same program scales transparently to GPUs with more or fewer SMs.

(b) nblocks = (3000 + 255) / 256 = **12** blocks → 12 × 256 = **3072** threads; **72** threads fail the `i < n` test. Thread 17 of block 9 → i = 9 × 256 + 17 = **2321**.

(c) kernelF: 3 × 2 = 6 blocks × (5 × 3 = 15) threads = **90** threads. kernelG: 4 × 6 = **24** threads. **No** — the inter-kernel synchronization barrier means kernelG starts only after all of kernelF's blocks have completed.

Lesson 36 — Final Integration: Whole RS Part

Ties together the Chapter 4 Part 1, Chapter 4 Part 2, Chapter 5 and GPU notes. Lesson/figure numbers refer to those files.

EXAM FOCUS Recent papers mix chapters inside one question (e.g. 2023-24 Q4: GPU system + dual-issue scheduling + CUDA) and include "**eight great ideas**" matching questions (2020-21 Q1, 2015-16, 2014-15). Use this lesson for the last revision pass.

1. The Eight Great Ideas — Matched to TF Statements

Great idea	Where it appears in the RS part
Design for Moore's Law	All cache levels now integrated on the processor chip; GPU parallelism doubling every 12–18 months
Use abstraction to simplify design	Procedures/functions; ISA as the hardware–software interface; CUDA's thread/block/grid
Make the common case fast	PC-relative branches (most branches are near); one-cycle simple instructions; not slowing every instruction to speed up jumps (Ch 4 P2, jump question)
Performance via parallelism	Separate I- and D-caches/memories (two accesses per cycle); fast multiplier with 31 adders; multiple issue; **GPUs**
Performance via pipelining	5-stage pipeline; **code reordering** to avoid load-use stalls; inefficient single-cycle design replaced by pipelining
Performance via prediction	Branch prediction (1-bit/2-bit, BTB); **prefetching** blocks before they are referenced
Hierarchy of memories	Registers → L1 → L2 → DRAM → disk; virtual memory; TLB
Dependability via redundancy	ECC on disk sectors; RAID

EXAM-READY ANSWER **TF statements (2020-21 Q1):** (a) "Code reordering is done to avoid the use of a load result in the next instruction" → **performance via pipelining** (it removes load-use stalls so the pipeline completes one instruction per cycle). (b) "Different caches for data and memory [i.e. instructions] can effectively increase bandwidth" → **performance via parallelism** (instruction fetch and data access proceed in the same cycle; also an instance of the memory hierarchy). (c) "GPUs have gained popularity as CPUs hit the power limit" → **performance via parallelism**.

2. One-Page Formula Sheet

Topic	Formula / rule
Next PC	PC + 4 · branch: PC + 4 + 4 × imm · jump: {(PC + 4)[31:28], imm26, 00}
Single-cycle clock	Longest instruction path (lw)
Pipelined cycles	5 + (n − 1) + stalls · speedup → T_{single} / T_{stage}
Stalls with forwarding	ALU → use: 0 · lw → next: 1 · branch in ID: ALU just before 1, lw just before 2 · taken branch: +1
Forwarding	EX hazard → 10 (EX/MEM) · MEM hazard → 01 (MEM/WB) only if not EX hazard · RegWrite and Rd ≠ 0
Load-use detect	ID/EX.MemRead and ID/EX.Rt ∈ {IF/ID.Rs, IF/ID.Rt}
Dual issue	IPC = instructions / cycles (no nops)
Cache fields	offset = $\log_2$ B · index = $\log_2$ sets · tag = rest
Total cache bits	blocks × (data + tag + valid + extra)
CPI with caches	base + Σ misses/instr × penalty · multilevel: + $m_{L1}\ t_{L2}$ + $m_{global}\ t_{mem}$

Topic	Formula / rule
AMAT	hit time + miss rate × miss penalty
Page table	$2^{(VA - offset)}$ × PTE size · inverted: physical pages × entry size
Disk	seek + ½ rotation + size/rate + controller
CUDA	i = blockIdx.x × blockDim.x + threadIdx.x · nblocks = [n / blockSize]

3. Diagram Checklist — Can You Draw These from Memory?

- Single-cycle datapath with control (Ch 4 P1, slide 26) and **with jump** (slide 32); the **bne / jr / jal / lui** additions.
- Branch hardware: sign-extend → shift-left-2 → adder with PC + 4; Branch AND Zero → PCSrc.
- **Pipeline registers** with their contents; pipelined datapath with control (Ch 4 P2, slide 52).
- **Forwarding unit** with ForwardA/ForwardB muxes (slide 65) and the **hazard detection unit** (slide 75).
- Multi-cycle pipeline diagrams with stalls, bubbles and flushes; **2-bit predictor FSM**.
- **Direct-mapped** cache address subdivision; **4-way set-associative** organization (Ch 5, slide 44).
- **Page-table translation** (slide 60); **TLB + cache** (slide 69); **TLB–cache flowchart** (slide 70).
- **Snooping invalidation table** (slide 85).
- **Intel / AMD discrete-GPU PCs** (GPU slide 15); unified GPU architecture (slide 23); **grid** → **block** → **thread** with local/shared/global memory.

4. Mixed Practice

Part (a): Constructed (combines Ch 4 P2 and Ch 5). Part (b): Constructed. Part (c): Adapted from the eight-great-ideas questions (2015-16, 2014-15).

(a) A 5-stage pipeline with forwarding has an ideal CPI of 1. 20 % of instructions are loads, and half of them are followed immediately by an instruction that uses the loaded value. 15 % are branches (decided in ID), 60 % of which are taken with a 1-cycle penalty. The I-cache misses 2 % of fetches, the D-cache misses 5 % of the 30 % loads/stores, and every miss costs 100 cycles. Find the effective CPI. Which single change would help most?

(b) A lw in the MEM stage misses in the D-cache. What happens to the pipeline? How is this different from a load-use hazard?

(c) Name the great idea behind each: (i) prefetching blocks into the cache before they are referenced; (ii) a multiplier that uses 31 adders instead of one adder 31 times; (iii) branch addresses relative to PC + 4; (iv) all levels of cache now integrated on the processor chip.

✓ Solution

(a)

$$\text{CPI} = 1 + 0.20 \times 0.5 \times 1 + 0.15 \times 0.6 \times 1 + 0.02 \times 100 + 0.30 \times 0.05 \times 100$$
$$= 1 + 0.10 + 0.09 + 2.0 + 1.5 = \mathbf{4.69}$$

Pipeline hazards add only 0.19 cycles; **memory stalls add 3.5**. The most effective change is to cut the **miss penalty** (e.g. an **L2 cache**) or the miss rates — not to improve the pipeline.

(b) On a cache miss the whole pipeline **stalls** (all stages freeze) for the miss penalty while the block is fetched from the next level; then execution continues with the data. A load-use hazard is a **1-cycle** bubble inserted by the hazard detection unit **even when the load hits**, because the value is not ready until the end of MEM.

(c) (i) **Performance via prediction.** (ii) **Performance via parallelism.** (iii) **Make the common case fast** (most branch targets are nearby). (iv) **Design for Moore's Law.**

Quick Notes — GPUs

Revision points for Lessons 33–35. Figure numbers point back to the lessons.

L33 · GPU Systems

- GPU = highly parallel, highly multithreaded processor for graphics + computing; **10^2–10^3×** the thread/data parallelism of a CPU; parallelism doubles every 12–18 months.
- Evolution: VGA controller → 3D acceleration (1990s) → "GPU" (2000) → programmable many-core (C/C++).
- **Heterogeneous system:** CPU (serial, latency) + GPU (parallel, throughput); split work by strengths.
- Historical PC: VGA controller on PCI bus with framebuffer memory; North Bridge (CPU, memory), South Bridge (legacy I/O).
- **Intel:** CPU –FSB– North Bridge (DDR2) –×16 PCIe– GPU (own memory) · **AMD:** North Bridge on CPU die –HyperTransport– chipset –×16 PCIe– GPU. ×16 PCIe = 16 GB/s (8 each way).
- **Memory access:** each can access the other's memory at lower bandwidth; GPU uses virtual addresses + its own **MMU**, page tables managed by the OS kernel; page attribute selects coherent / noncoherent PCIe transactions; CPU reaches GPU memory via an **aperture** in PCIe address space.
- UMA: shared system memory, cheaper, slower. Multi-GPU (SLI). Consoles: custom buses. APIs: OpenGL / Direct3D via driver; PCIe replaced AGP.

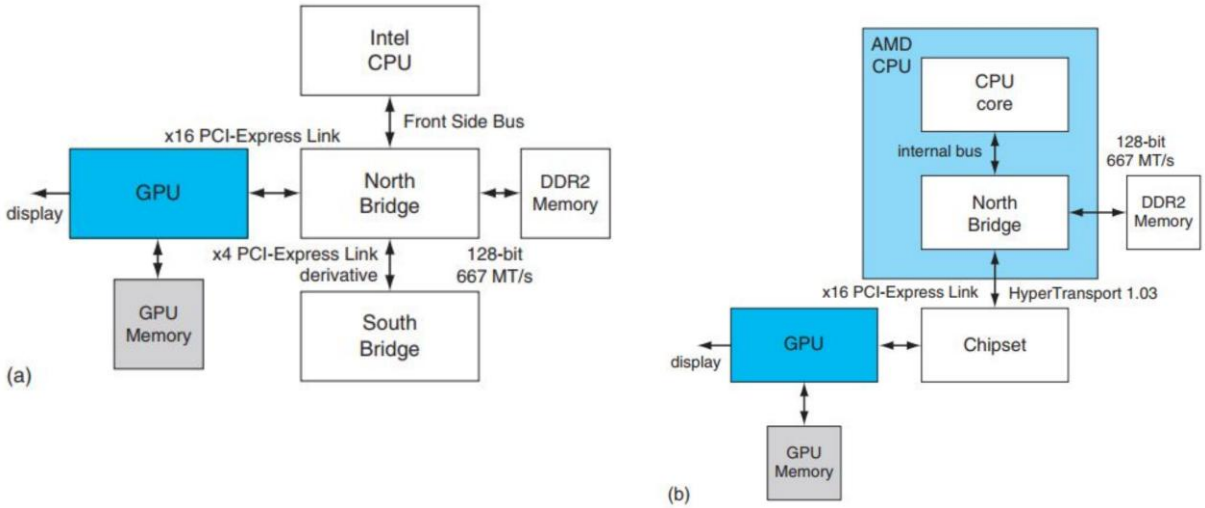
L34 · Pipeline and Architecture

- Logical pipeline: Input Assembler → **Vertex** → **Geometry** → Setup/Rasterizer → **Pixel** → Output Merger (programmable shaders in bold).
- HLSL / Cg: C-like, no pointers/recursion, independent parallel instances.
- **Unified architecture:** all shader types and computing run on one programmable processor array; logical pipeline recirculates through it.
- Tesla (slides): **112 SPs in 14 SMs**; SM = 8 SPs + 2 SFUs + shared memory + I/constant caches + MT issue; 96 threads per SP (768 per SM).
- SIMT: **warps of 32 threads**; issue unit picks a ready warp each cycle; divergence serializes paths.

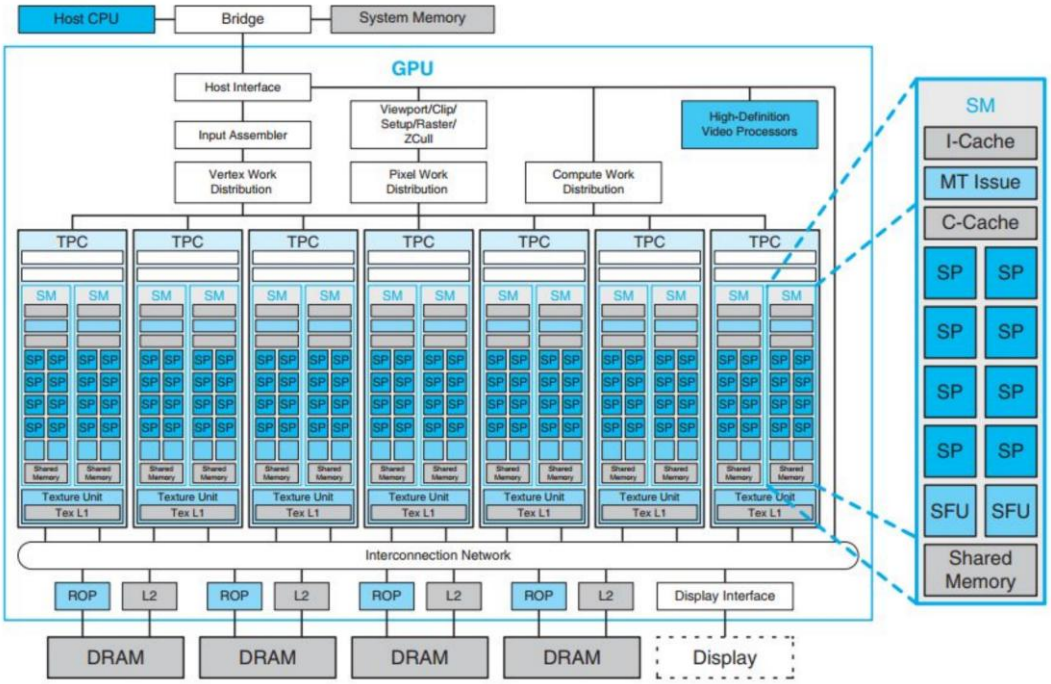
L35 · CUDA

- SPMD; **kernel** = code for one thread (`__global__`); **thread** (threadIdx) → **block** (shared memory + `__syncthreads()`, blockIdx) → **grid** (independent blocks, same kernel).
- Memory: local (thread) · shared (block, on-chip) · global (all, board DRAM).
- Blocks run in any order → transparent scalability; blocks set by data size.
- SAXPY: i = blockIdx.x × blockDim.x + threadIdx.x; nblocks = (n + 255)/256; `if (i < n)`.
- **Mapping:** grid → whole GPU (blocks distributed to free SMs) · block → **one SM** (shares its shared memory/barrier; several blocks per SM) · thread → **SP core** (own registers), scheduled in warps.

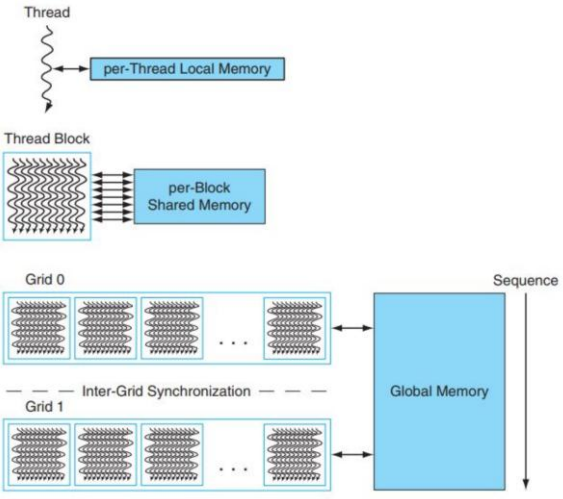
Key Diagrams to Reproduce



Contemporary PCs with Intel and AMD CPUs (slide 15) — same as Fig. 33.2



Basic unified GPU architecture (slide 23) — same as Fig. 34.2



Thread / block / grid and their memories (slide 33) — same as Fig. 35.2